# Einsatz von Large Language Models im Umfeld der Robotik hinsichtlich On-Premise-Lösungen

Moritz Jahn
Matrikelnummer: 3221169

23. März 2026

# Inhaltsverzeichnis

**Abkürzungsverzeichnis** **v**

**1 Einführung** **1**
1.1 Problemstellung . . . . . 2
1.2 Stand der Technik . . . . . 2
1.3 Zielsetzung . . . . . 4
1.4 Aufbau der Arbeit . . . . . 4

**2 Grundlagen Large Language Models** **5**
2.1 Tokenisierung und Embedding . . . . . 6
2.2 Der Transformer-Block . . . . . 7
2.3 Ausgabeschicht, Training und Quantisierung . . . . . 10
2.4 Zusammenfassung . . . . . 12

**3 Sprachverarbeitung: STT und TTS** **13**
3.1 Physikalische Eigenschaften von Audiosignalen . . . . . 13
3.1.1 Text-to-Speech (TTS) . . . . . 16
3.2 Das eingesetzte System . . . . . 16
3.3 Zusammenfassung . . . . . 18

**4 On-Premise-Infrastruktur und Hardware** **19**
4.1 Rechenoperationen eines LLMs nano-GPT als Beispiel . . . . . 19
4.1.1 Modellkonfiguration . . . . . 20
4.1.2 Rechenoperationen und Skalierung . . . . . 20
4.2 Motivation für On-Premise-Betrieb . . . . . 21
4.3 Hardwareanforderungen . . . . . 21
4.3.1 GPU und VRAM . . . . . 21
4.3.2 CPU und Arbeitsspeicher . . . . . 22
4.3.3 Speichersystem . . . . . 22
4.4 Laboraufbau am IZPL . . . . . 22
4.5 Zusammenfassung . . . . . 23

**5 Evaluation** **24**
5.1 Methodik der Evaluation . . . . . 24
5.2 Antwortzeiten und Performanz . . . . . 25
5.3 Qualität der Konversation . . . . . 26

**6 Fazit und Ausblick** **28**
6.1 Zusammenfassung der Ergebnisse . . . . . 28
6.2 Ausblick auf die Masterarbeit . . . . . 28

**A Anhang** **29**
A.1 Konfigurationsdateien . . . . . 29
A.2 Testprotokolle . . . . . 29
A.3 Zusätzliche Abbildungen . . . . . 29

**Literaturverzeichnis** **30**

# Abbildungsverzeichnis


1.1 Geschätzter VRAM-Bedarf aktueller LLM-Flagschiffmodelle . . . . . 3
1.2 Vergleich quelloffener LLMs: Chatbot Arena Elo vs. VRAM-Bedarf (INT4). Grün hervorgehobene Modelle sind für den Betrieb auf einer einzelnen Consumer-GPU ($\leq 48$ GB) geeignet. Datenquelle: onyx.app/self-hosted-llm-leaderboard, Stand März 2026. . . . . 4

2.1 Datenfluss eines Large Language Models . . . . . 5
2.2 Vektor-Embedding für den Satz „LLMs with Igus Rebel 6 DOF“ . . . . . 6
2.3 Beispielhafte Darstellung der Tokenisierung . . . . . 7

3.1 Datenfluss eines Large Language Models . . . . . 15

5.1 Simulierter VRAM-Bedarf je Quantisierungsstufe im Vergleich zum 12-GB-Limit der RTX 3080 Ti . . . . . 24
5.2 Generierungsgeschwindigkeit (Tokens/s) in Abhängigkeit der Quantisierung 25
5.3 Prefill-Latenz (Time-to-First-Token) über verschiedene Prompt-Längen (Q4_K_M) 25
5.4 Kumulative Token-Ausgabe über die Zeit bei der 4-Bit-Quantisierung . . . 26
5.5 Zeitlicher VRAM-Verlauf während der Modell-Ausführung gegliedert in Laden, Prefill und Generierung . . . . . 26
5.6 Qualitätsverlust (Perplexity auf WikiText-2) durch Quantisierungsverfahren 27

# Tabellenverzeichnis

2.1 VRAM-Bedarf eines 7B-Sprachmodells bei verschiedenen Quantisierungsstufen. . . . . 11

3.1 Architekturparameter – LLaMA 7B (fett) ist das in dieser Arbeit eingesetzte Modell. . . . . 17
3.2 Übersicht der eingesetzten Komponenten. . . . . 18

4.1 Konfiguration des nano-GPT-Modells [19]. . . . . 20
4.2 Vergleich Cloud-API vs. On-Premise-Betrieb. . . . . 21
4.3 Gängige GPUs für lokale LLM-Inferenz. ✓ : ausreichend, ✓✓ : gut geeignet, ✓✓✓ : optimal. . . . . 22
4.4 Hardware- und Software-Konfiguration am IZPL. . . . . 23

# Abkürzungsverzeichnis

**LLM** Large Language Model

**VLM** Visual Language Model

**NLP** Natural Language Processing

**GPU** Graphics Processing Unit

**CPU** Central Processing Unit

**VRAM** Video Random Access Memory

**RAM** Random Access Memory

**ROS** Robot Operating System

**API** Application Programming Interface

**TTS** Text-to-Speech

**STT** Speech-to-Text

**RAG** Retrieval-Augmented Generation

**GGUF** GPT-Generated Unified Format

# Kapitel 1

# Einführung

Kaum ein anderes Thema hat die Technologiebranche in den letzten Jahren so sehr geprägt wie künstliche Intelligenz. Um im globalen Wettbewerb um die leistungsfähigsten Sprachmodelle bestehen zu können, investieren Unternehmen Milliardensummen in Recheninfrastruktur. So nahm xAi, die Firma von Elon Musk, erst 2024 einen Cluster mit über 100.000 NVIDIA HGX H100 GPUs in Betrieb. Eine dieser GPUs hat einen Wert von 40.000 US-Dollar. Das in Tennessee befindliche Cluster stellt den bislang größten KI-Supercomputer der Welt dar und dient dem Training des generativen Modells Grok sowie zur Bereitstellung der Rechenkapazitäten für dessen Betrieb. [1]. Nicht nur xAi, sondern auch Meta, Google und OpenAI haben ähnlichen Zugang zu solchen großen Clustern. Die für solche Systeme benötigte Hardware stammt nahezu ausschließlich vom Branchenführer NVIDIA, der über 90 % des Marktes für Datenzentren-GPUs hält. CEO Jensen Huang bezifferte das Auftragsvolumen des Unternehmens für die Jahre 2025 und 2026 auf rund 500 Milliarden US-Dollar. [2] Dieser massive Ausbau von Rechenzentren hat auch weitreichende energetische Konsequenzen: Laut einer Prognose der Internationalen Energieagentur (IEA) könnte sich der weltweite Stromverbrauch von Rechenzentren bis 2030 verdoppeln, wobei der Bedarf an GPU-Clustern, wie sie für KI-Workloads eingesetzt werden, im Zeitraum 2025 bis 2030 um 225 % steigt [3].

Angesichts dieser Entwicklungen stellt sich die Frage, ob der Betrieb solcher ressourcenintensiver Modelle für jeden Anwendungsfall notwendig ist. Insbesondere in industriellen Einsatzszenarien etwa auf Edge-Geräten oder Robotern in der Produktion, die häufig keine dauerhafte Internetverbindung wünschen oder zulassen bieten sich deutlich kleinere, spezialisierte Modelle an. Diese können On-Premise, also vor Ort auf eigener Hardware, betrieben werden und erfordern weder die enormen Rechenkapazitäten cloudbasierter Infrastrukturen noch den damit verbundenen Energieverbrauch. Die vorliegende Arbeit untersucht daher, inwieweit solche lokalen Modelle für spezifische Anwendungsfälle eine praktikable und effiziente Alternative zu den großen, cloudbasierten Sprachmodellen

darstellen.

## 1.1 Problemstellung

Wie bereits erwähnt, wirft der rasante Ausbau cloudbasierter KI-Dienste neben technischen und energetischen Fragen zunehmend auch Bedenken hinsichtlich Datenhoheit, Sicherheit und technologischer Unabhängigkeit auf. Die dominierenden KI-Plattformen, darunter Dienste von OpenAI, Google und Anthropic, werden überwiegend in Rechenzentren in den USA betrieben. Europäische Konkurrenz im Bereich der führenden Sprachmodelle gibt es kaum. Daten, die an diese Modelle übermittelt werden, verlassen damit häufig den europäischen Rechtsraum und unterliegen potenziell dem Zugriff ausländischer Behörden, etwa im Rahmen des US-amerikanischen CLOUD Act. Laut dem Bitkom Cloud Report 2025 sehen 78 % der deutschen Unternehmen eine zu starke Abhängigkeit von US-Cloud-Anbietern und für 97 % der befragten Cloud-Nutzer spielt die Herkunft des Anbieters bei der Auswahl eine wesentliche Rolle. [4].

Für produzierende Unternehmen verschärft sich diese Problematik zusätzlich: Sensible Produktionsdaten, Prozessparameter oder Qualitätsinformationen dürfen aus Gründen des Schutzes von Geschäftsgeheimnissen und zur Sicherstellung der Compliance häufig das Unternehmensnetzwerk nicht verlassen. Roboter und Edge-Geräte in der Fertigung operieren zudem oft in Umgebungen ohne zuverlässige oder verfügbare Internetanbindung. Die Nutzung cloudbasierter KI-Modelle ist in solchen Szenarien daher entweder nicht möglich oder mit erheblichen Risiken verbunden. Somit stellt sich die Frage, ob kleinere, spezialisierte Sprachmodelle, die vollständig vor Ort, also On-Premise, betrieben werden können, eine sichere, unabhängige und zugleich leistungsfähige Alternative für industrielle Anwendungsfälle darstellen. Bevor auf diese Frage eingegangen wird, wird im folgenden Kapitel der Stand der Technik erläutert.

## 1.2 Stand der Technik

Wie im vorherigen Kapitel bereits angesprochen, beschäftigt sich diese Arbeit primär mit der Verwendung von Large Language Models (LLM) auf Edge-Geräten und in On-Premise-Anwendungen. In diesem Kapitel wird deshalb zuerst der aktuelle Entwicklungsstand der führenden LLM-Modelle dargelegt, um den Unterschied zwischen diesen und kleinen, on Edge Modellen zu verdeutlichen.

Der Begriff des maschinellen Lernens ist in der Informatik keineswegs neu. Bereits seit Anfang des 21. Jahrhunderts existieren Algorithmen, die nichtlineare Zusammenhänge in Daten erkennen können. Einen entscheidenden Durchbruch markierte das Jahr 2017, als ein Team um Vaswani von OpenAI die sogenannte Transformer-Architektur vorstellte.

Diese Architektur ermöglichte den Übergang von sequenzieller zu paralleler Datenverarbeitung und erlaubte es, diese Parallelisierung mit der massiven Rechenleistung moderner GPUs zu koppeln, um deutlich größere Datenmengen effizient zu verarbeiten.

Auf dieser Grundlage sind Modelle entstanden, die komplexe Muster in umfangreichen Textdaten erkennen und natürlichsprachliche Ausgaben erzeugen können: sogenannte Large Language Models (LLMs). Laut Kasneci ist ein LLM ein KI-System, das auf einer riesigen Menge von Textdaten trainiert wurde, um natürliche Sprache zu interpretieren und menschenähnliche Antworten auf textbasierte Eingabeaufforderungen oder Fragen zu generieren. [**llm_definition**].

Seither hat sich die Leistungsfähigkeit dieser Modelle rasant weiterentwickelt. Dabei ist die Anzahl der Parameter und somit auch der Ressourcenbedarf stetig gestiegen. Abbildung 1.1 zeigt die geschätzten Mengen an schnell zugänglichem Grafikspeicher (VRAM), die für den Betrieb aktueller Flaggschiffmodelle erforderlich sind. Für Edge-Geräte, die auf eine begrenzte Speicherkapazität angewiesen sind, kommen diese Modelle daher nicht infrage.

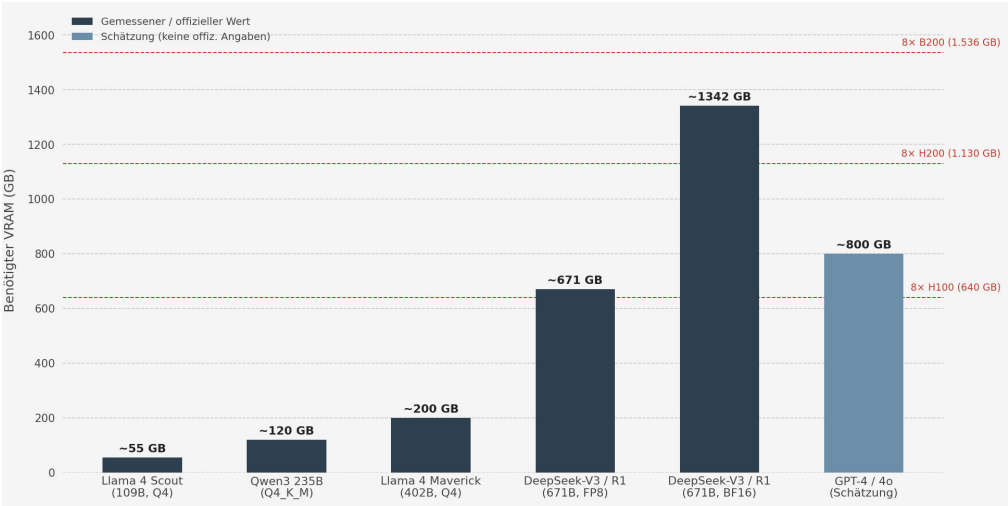


Abbildung 1.1: Geschätzter VRAM-Bedarf aktueller LLM-Flagschiffmodelle

Die leistungsstärksten quelloffenen Modelle, die für einen On-Premise-Betrieb relevant sind, werden in Abbildung 1.2 gegenübergestellt. Dabei wird der Chatbot-Arena-Elo-Score, ein Crowdsourcing-basiertes Maß für die vom Nutzer wahrgenommene Antwortqualität, dem jeweiligen VRAM-Bedarf bei 4-Bit-Quantisierung (INT4) gegenübergestellt. Modelle, die mit maximal 48 GB VRAM auskommen, lassen sich bereits auf einer einzelnen Consumer-GPU betreiben und sind somit für On-Premise-Szenarien besonders interessant. Grundsätzlich sind die jeweils neuesten Modellgenerationen die leistungsstärksten. Zum Zeitpunkt der Erstellung dieser Arbeit ist Qwen 3.5 von Alibaba mit einem relativen Chatbot-Arena-Elo-Score von 1450 das leistungsstärkste quelloffene Modell.

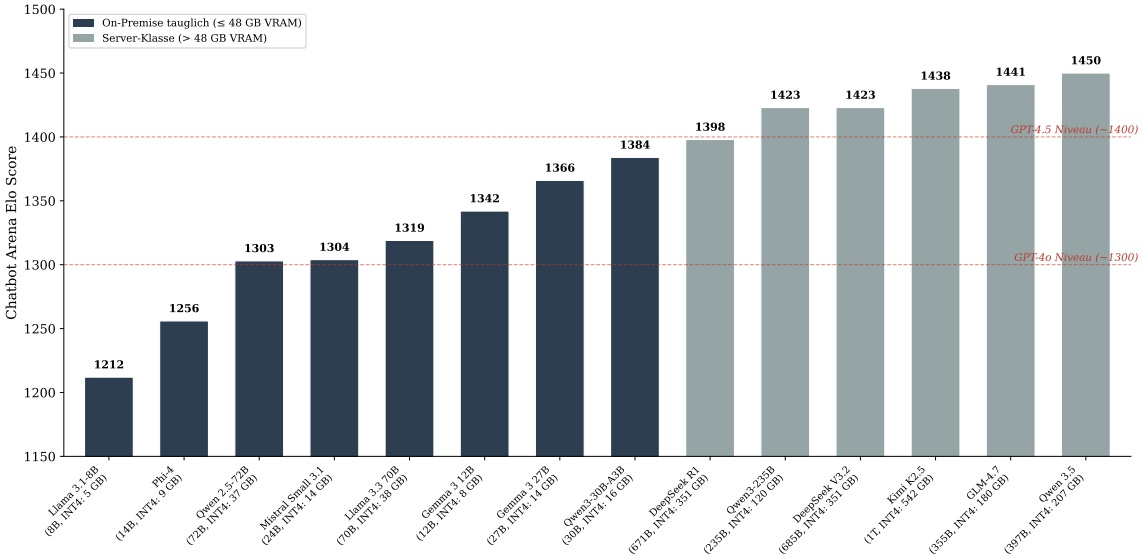


Abbildung 1.2: Vergleich quelloffener LLMs: Chatbot Arena Elo vs. VRAM-Bedarf (INT4). Grün hervorgehobene Modelle sind für den Betrieb auf einer einzelnen Consumer-GPU ($\leq 48$ GB) geeignet. Datenquelle: onyx.app/self-hosted-llm-leaderboard, Stand März 2026.

## 1.3 Zielsetzung

Im Rahmen dieser Forschungsarbeit wird der Einsatz von On-Premise-LLMs am Beispiel der Anwendungsmöglichkeit im Innovationszentrum für Produktion und Logistik (im folgenden IZPL) der OTH Regensburg untersucht. Dabei wird auf öffentlich verfügbare Modelle zurückgegriffen, die lokal auf einem Server des IZPL betrieben werden. Ziel ist es, die Grundlagen, technischen Anforderungen und Möglichkeiten solcher Systeme für künftige robotiknahe Anwendungen zu analysieren. Konkret dient diese wissenschaftliche Arbeit als Fundament für eine Abschlussarbeit im IZPL Regensburg.

## 1.4 Aufbau der Arbeit

Zunächst erfolgt eine Einführung in die mathematischen Konzepte, auf denen die Modelle basieren. Die im Folgenden aufgeführten Hardwarespezifikationen zeigen, welche Computing-Kapazitäten erforderlich sind, um „künstliche Intelligenzen" (KI) zu betreiben. Nach der Evaluation einer Testanwendung in Kapitel 4 werden Sicherheits- und Fehlerbilder aufgezeigt. Die vorliegende Arbeit schließt mit Kapitel 6, das einen Ausblick auf zukünftige Forschungsgebiete gibt und ein Fazit zieht.

# Kapitel 2

# Grundlagen Large Language Models

Dieses Kapitel legt die theoretischen Grundlagen für das Verständnis moderner Large Language Models (LLMs). Es werden die mathematischen Grundlagen vom Embedding bis zur Softmax-Funktion erläutert sowie die Konzepte des Trainings und der Quantisierung vorgestellt. Das Kapitel beginnt mit Abbildung 2.1, welche sich auf den grundlegenden Mechanismus eines LLMs fokussiert, den Transformer.

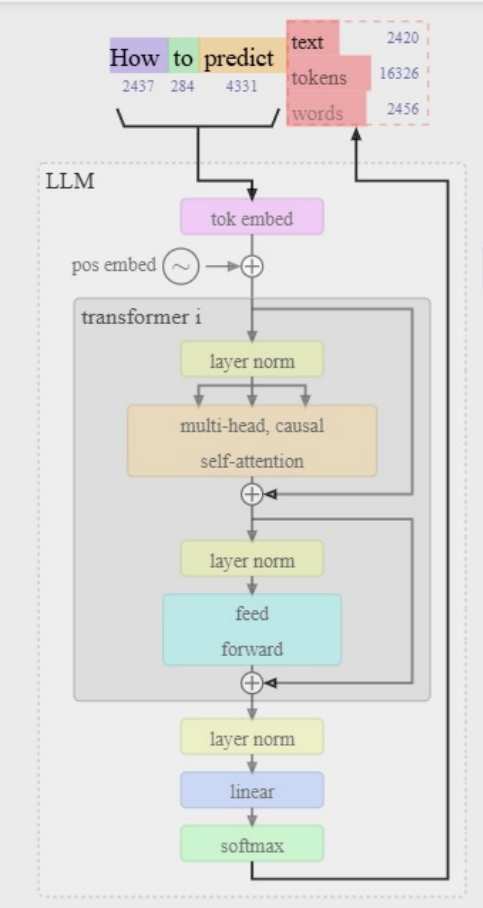


Abbildung 2.1: Datenfluss eines Large Language Models

Abbildung 2.1 veranschaulicht den grundlegenden Datenfluss eines LLMs. Zusammenge-

fasst verabeitet das Modell Eingabetext in und berechnet am Ende Wahrscheinlichkeiten, um das nächste logische Wort vorherzusagen.

## 2.1 Tokenisierung und Embedding

Bevor jedoch der Transformer zum Einsatz kommt, ist der erste Schritt eines jeden LLMs das sogenannte Tokenisieren. Ein Token im Kontext des maschinellen Lernens wurde im Januar dieses Jahres von Daniel Jurafsky & James H. Martin wie folgt definiert: ¨Tokens sind Einheiten, welche entstehen, nachdem wir Algorithmen wie BPE oder WordPiece anwenden, um Texteingaben für LLMs vorzubereiten.“[5] In Abbildung 2.2 wird dieser Schritt mit dem ersten Block dargestellt und heißt übergeordnet Embedding. Beim Tokenisieren werden Wörter in Arrays umgewandelt. Zur Einordnung: Einer der ersten LLMs der Firma OpenAI, GPT-2, benutzt 768 Koordinaten, um den multidimensionalen Vektor eines einzigen Tokens aufzuspannen und um dessen Kontext zu speichern (siehe 2.1 in [6]). In Abbildung 2.2 ist das Vektor-Embedding für den Satz ¨LLMs with Igus Rebel 6 DOF“ mithilfe von Hauptachsentransformation (englisch Principal Component Analysis, kurz PCA) im dreidimensionalen Raum für GPT-2 dargestellt. Zur Tokenisierung wurde der bis heute standardmäßig verwendete Tokenisierungsalgorithmus Byte Pair Encoding verwendet.

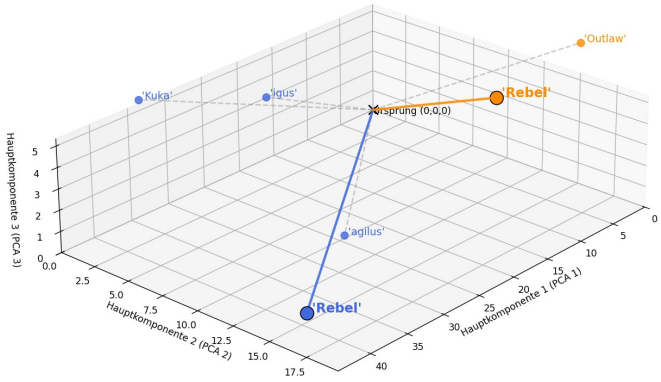


Abbildung 2.2: Vektor-Embedding für den Satz „LLMs with Igus Rebel 6 DOF“

**Byte Pair Encoding (BPE)** ist ein datengetriebener Teilwort-Tokenizer. Der Algorithmus beginnt mit einem Lookup-Tabel als Trainingskorpus und fasst iterativ das häufigste benachbarte Zeichenpaar,welche er im Trainigskorpuse nachschaut, zu einem neuen Token zusammen. Dieser Prozess wiederholt sich, bis eine gewünschte Vokabulargröße $|\mathcal{V}|$ erreicht ist [7]. Ursprünglich in den 90er Jahren für Speicherplatzreduzierung entwickelt zerlegt er seltene oder zusammengesetzte Wörter in häufige Teilwörter und kann somit prinzipiell jede Zeichenkette verarbeiten. Häufige Wörter erhalten dabei meist ein einzelnes Token, seltene Wörter oder Pluralformen werden in mehrere Tokens aufgeteilt.

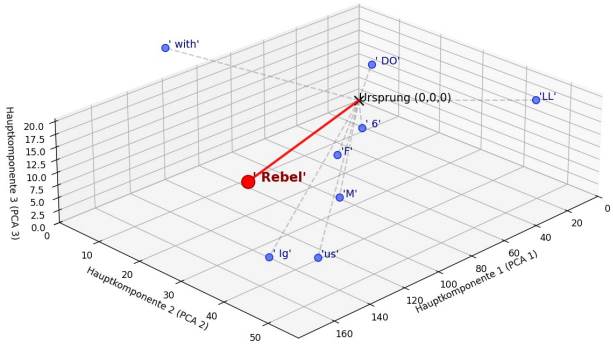


Abbildung 2.3: Beispielhafte Darstellung der Tokenisierung

Statische Embeddings wie BPE haben jedoch ein grundlegendes Problem: Jedes Wort erhält genau einen festen Vektor, unabhängig vom Kontext. In Abbildung 2.2 ist dies am Wort Rebel dargestellt, das sowohl den igus-Leichtbauroboter als auch einen Aufständischen bezeichnen kann. Im Roboter-Kontext zeigt der Vektor in ähnliche Richtungen wie igus kuka und agilus. Im Kontext eines Rebellen hingegen in Richtung eines Verwandten wortes wie Outlaw. Diese Einordnungen hat das Modell während des Trainings gelernt. Welcher Kontext gemeint ist im Prompt kann ein statisches Embedding nicht Unterscheiden erst der im folgenden Abschnitt vorgestellte Transformer löst dieses Problem.

Token-Embedding:

$$x_i = e_i + p_i \tag{2.1}$$

## 2.2 Der Transformer-Block

Die Transformer-Architektur ist ein Modell, das fundamental auf dem Self-Attention-Mechanismus beruht. In seiner ursprünglichen Form besteht ein Transformer aus einem Encoder und einem Decoder: Der Encoder berechnet kontextbezogene Repräsentationen der Eingabe, der Decoder nutzt diese, um autoregressiv das jeweils nächste Token vorherzusagen. Moderne LLMs wie LLaMA oder GPT verwenden vorwiegend den Decoder-Teil.

Ein einzelner Transformer-Block ist die zentrale Verarbeitungseinheit und besteht aus zwei Hauptoperationen: *Multi-Head Self-Attention* und einem *Feed-Forward-Netzwerk* (FFN), jeweils kombiniert mit einer Normierungsschicht und einer Residualverbindung:

$$x' = x + \text{Attention}\big(\text{Norm}(x)\big) \tag{2.2}$$

$$x'' = x' + \text{FFN}\big(\text{Norm}(x')\big) \tag{2.3}$$

Durch das $N$-fache Hintereinanderschalten solcher Blöcke entsteht eine tiefe Architektur, die schrittweise zunehmend abstrakte Repräsentationen aufbaut. Die einzelnen Bestandteile werden im Folgenden erläutert.

**Normierungsschicht.** Zur Stabilisierung des Trainings tiefer Netze wird vor jeder Hauptoperation eine Normierungsschicht angewendet. Die klassische *Layer Normalization* berechnet Mittelwert $\mu$ und Standardabweichung $\sigma$ über die Feature-Dimension:

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \quad \mu = \frac{1}{d} \sum_{i=1}^{d} x_i \tag{2.4}$$

Eine vereinfachte Variante ist die **RMSNorm** (Root Mean Square Normalization), die den Mittelwert nicht subtrahiert und damit rechnerisch effizienter ist:

$$\text{RMSNorm}(x) = \gamma \cdot \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^{d} x_i^2 + \epsilon}} \tag{2.5}$$

Die Normierung verhindert, dass Aktivierungswerte über viele Schichten hinweg exponentiell wachsen oder verschwinden (Vanishing/Exploding Gradients).

**Attention – Intuition.** Grundidee des Attention-Mechanismus: Gegeben eine Anfrage (Query $q$), soll ermittelt werden, welchem Schlüssel (Key $k$) die Anfrage die meiste Aufmerksamkeit schenken soll, um den zugehörigen Wert (Value $v$) abzufragen. In natürlicher Sprache bedeutet dies: Wenn ein Token wie „Bär“ verarbeitet wird, berechnet der Mechanismus, wie stark es auf Kontextwörter wie „Teddy“ oder „süß“ achten soll, um seine eigene Bedeutung im Satz korrekt zu erfassen.

Formal erzeugen drei lineare Projektionen für jedes Token $x_i$ die Vektoren Query $q_i$, Key $k_i$ und Value $v_i$:

$$q_i = W_Q\, x_i, \quad k_i = W_K\, x_i, \quad v_i = W_V\, x_i \tag{2.6}$$

Die **Scaled Dot-Product Attention** berechnet Ähnlichkeitsgewichte zwischen Query und allen Keys und aggregiert die Values entsprechend gewichtet:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V \tag{2.7}$$

Die Division durch $\sqrt{d_k}$ skaliert die Skalarprodukte, um numerisch stabile Softmax-Werte zu gewährleisten. Die **kausale Maske** $M$ stellt sicher, dass Token $t_i$ nur auf vorherige

Positionen $t_j$ mit $j \leq i$ zugreifen kann:

$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases} \tag{2.8}$$

**Multi-Head Attention (MHA).** Eine einzelne Attention-Berechnung kann nur einen Aspekt des Kontexts erfassen. Beim **Multi-Head Attention** werden daher $h$ parallele Attention-Berechnungen in Unterräumen der Dimension $d_k = d/h$ ausgeführt und anschließend konkateniert. Die Rückprojektion in den Ausgaberaum erfolgt mit der Matrix $W_O$:

$$\mathrm{MultiHead}(Q, K, V) = \left[\mathrm{head}_1 \parallel \cdots \parallel \mathrm{head}_h\right] W_O \tag{2.9}$$

Jeder Kopf verfügt über eigene Gewichtsmatrizen $W^Q$, $W^K$, $W^V$ und kann somit unterschiedliche Aspekte des Kontexts erlernen (z. B. syntaktische Abhängigkeiten, semantische Ähnlichkeit oder Koreferenzen). Varianten wie *Grouped-Query Attention* (GQA) und *Multi-Query Attention* (MQA) reduzieren den Rechenaufwand, indem Key- und Value-Matrizen über mehrere Attention-Köpfe hinweg geteilt werden.

**Feed-Forward-Netzwerk.** Nach der Self-Attention verarbeitet ein positionsweises Feed-Forward-Netzwerk jede Token-Position unabhängig. In der ursprünglichen Transformer-Formulierung besteht es aus zwei linearen Transformationen mit einer nichtlinearen Aktivierung dazwischen [8]:

$$\mathrm{FFN}(x) = W_2 \cdot \mathrm{ReLU}(W_1\, x + b_1) + b_2 \tag{2.10}$$

Die innere Dimension $d_{\mathrm{ff}}$ ist typischerweise $4 \cdot d$. Das FFN speichert einen wesentlichen Teil des Faktenwissens des Modells.

**Rotary Position Embeddings (RoPE).** Neuere Architekturen verwenden funktionale Kodierungen, die die Positionsinformation direkt in den Attention-Mechanismus einbetten. **Rotary Position Embeddings (RoPE)** kodieren die Position durch eine Rotation des Query- und Key-Vektors mit einem positionsabhängigen Winkel [9]. Für einen zweidimensionalen Ausschnitt gilt:

$$\mathrm{RoPE}(x, i) = \begin{pmatrix} x_1 \cos(i\,\omega) - x_2 \sin(i\,\omega) \\ x_1 \sin(i\,\omega) + x_2 \cos(i\,\omega) \end{pmatrix}, \quad \omega_k = \frac{1}{10000^{2k/d}} \tag{2.11}$$

Da das Skalarprodukt zweier rotierter Vektoren nur von ihrer *relativen* Position abhängt, kodiert RoPE die Relativdistanz direkt im Attention-Mechanismus.

**Residualverbindungen.** Residualverbindungen (Skip Connections) addieren das Ein-

gangssignal direkt zur Ausgabe einer Operation und lösen so das Problem der verschwindenden Gradienten in tiefen Netzen:

$$\text{out} = f(x) + x \tag{2.12}$$

Jeder Transformer-Block besitzt zwei Residualverbindungen: eine nach der Self-Attention und eine nach dem FFN.

## 2.3 Ausgabeschicht, Training und Quantisierung

Nach dem letzten Transformer-Block wird die Ausgabe des letzten Tokens durch eine Normierung und eine lineare Projektion auf die Vokabulargröße abgebildet:

$$z = W_U \cdot \text{Norm}\big(x_T^{(N)}\big), \quad z \in \mathbb{R}^{|\mathcal{V}|} \tag{2.13}$$

Der resultierende Vektor $z$ (**Logits**) wird durch **Softmax** in eine Wahrscheinlichkeitsverteilung über das gesamte Vokabular umgewandelt:

$$P\big(t_{T+1} = w \mid t_1, \ldots, t_T\big) = \frac{\exp(z_w/\tau)}{\sum_j \exp(z_j/\tau)} \tag{2.14}$$

Der **Temperature**-Parameter $\tau$ steuert die Schärfe der Verteilung: kleine Werte ($\tau < 1$) machen das Modell deterministischer, große Werte ($\tau > 1$) erhöhen die Vielfalt. Das gewählte Token wird an die Sequenz angehängt und der Prozess wiederholt (**autoregressive Generierung**), bis ein Stopptoken erreicht ist. Gängige Sampling-Strategien sind **Top-$k$ Sampling** (nur die $k$ wahrscheinlichsten Tokens) und **Top-$p$ (Nucleus) Sampling** (kleinste Menge mit kumulativer Wahrscheinlichkeit $\geq p$).

**Pretraining.** Beim Pretraining lernt das Modell, das jeweils nächste Token einer Text-Sequenz vorherzusagen (**Causal Language Modeling, CLM**). Die Verlustfunktion ist die mittlere negative Log-Likelihood (Cross-Entropy) über alle Positionen:

$$\mathcal{L}_{\text{CLM}}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \log P(w_t \mid w_1, \ldots, w_{t-1}; \theta) \tag{2.15}$$

Die Gewichte $\theta$ werden durch stochastischen Gradientenabstieg (Adam-Optimizer) minimiert. Das Pretraining auf großen Textkorpora befähigt das Modell, allgemeines Weltwissen und sprachliche Strukturen zu erlernen.

**Finetuning und RLHF.** Nach dem Pretraining ist ein Modell nur auf Token-Vervollständigung optimiert. Durch **Supervised Fine-Tuning (SFT)** auf Paaren aus Anweisung und ge-

wünschter Antwort wird das Modell auf Konversationen spezialisiert. **Reinforcement Learning from Human Feedback (RLHF)** verfeinert das Modell weiter: Menschliche Bewerter vergleichen Modellantworten, daraus wird ein *Reward Model* trainiert, und das Sprachmodell wird so optimiert, dass sein Reward maximiert wird.

**Quantisierung.** Modellgewichte werden standardmäßig als 16-Bit-Gleitkommazahlen gespeichert. Auf lokaler Hardware mit begrenztem VRAM ist dies für große Modelle oft nicht praktikabel. **Quantisierung** ist eine Technik, welche die numerische Präzision neuronaler Netzwerk-Gewichte reduziert, um die Rechenkosten und den Speicherbedarf bei der Inferenz zu senken [10]. Der theoretische VRAM-Bedarf lässt sich durch die Reduktion der Bitbreite stark minimieren:

$$\mathrm{VRAM}_{\mathrm{min}} \approx \frac{N_{\mathrm{Parameter}} \times \mathrm{Bitbreite}}{8 \times 10^9} \; [\mathrm{GB}] \tag{2.16}$$

Obwohl die INT8-Quantisierung derzeit aufgrund eines guten Verhältnisses zwischen Inferenzleistung und Modellgenauigkeit weit verbreitet ist, wird ihre allgemeine Akzeptanz durch teilweise auftretende Ausreißer in den Aktivierungswerten limitiert. Zwar stellt FP8 einen vielversprechenden neuen Datentyp dar, dieser wird jedoch bisher durch fehlende Hardware-Verfügbarkeit eingeschränkt [10].

Infolgedessen gewinnt die sogenannte *Weight-Only Quantization* stark an Popularität. Bei diesem Ansatz werden lediglich die Gewichte in niedriger Präzision (z. B. 4-Bit) quantisiert, während für die Aktivierungen die höhere Präzision (z. B. 16-Bit-Gleitkommazahlen) beibehalten wird. Dies gewährleistet, dass die Genauigkeit des Modells erhalten bleibt. Insbesondere Ansätze der 4-Bit-Weight-Only-Quantisierung haben sich bei der Inferenz von LLMs als äußerst effektiv erwiesen [10].

| Format | Bitbreite | VRAM (7B-Modell) | Qualitätsverlust |
|---|---|---|---|
| FP16 | 16 Bit | ≈14 GB | Referenz |
| INT8 | 8 Bit | ≈7 GB | minimal |
| Q4_K_M | 4 Bit | ≈4 GB | gering, praxistauglich |
| Q2_K | 2 Bit | ≈2 GB | deutlich spürbar |

Tabelle 2.1: VRAM-Bedarf eines 7B-Sprachmodells bei verschiedenen Quantisierungsstufen.

Die Open-Source-Community greift solche niederbittigen Quantisierungsmethoden intensiv auf. Eine führende Implementierung stellt das C++-basierte Framework `llama.cpp` dar, welches auf der GGML-Bibliothek aufbaut. Solche Implementierungen sind durch spezifische Backends (wie OpenBLAS, CUDA, Metal, OpenCL) extrem optimiert und

erreichen durch diese Co-Optimierung von Hard- und Software Höchstleistungen [10]. Das dabei eingesetzte Quantisierungsformat **GGUF** (bzw. GGML) verwendet eine Namenskonvention nach dem Muster `qX_Y`. Hierbei steht `X` für die zur Gewichtsspeicherung verwendete Bitbreite. Der Wert `Y` gibt die Art der Quantisierung an: `0` kennzeichnet eine symmetrische Quantisierung ohne Nullpunkt, während `1` für eine asymmetrische Quantisierung steht, bei der jede Skalierung über einen Nullpunkt verfügt. Es wird im Allgemeinen nicht empfohlen, Formate wie `q2` oder `q3` zu verwenden, da hierbei die Inferenzgenauigkeit des Modells massiv abfällt [10].

## 2.4 Zusammenfassung

Dieses Kapitel hat die theoretischen Grundlagen der LLM-Architektur erläutert – von der Tokenisierung und dem Embedding über den Transformer-Block mit Self-Attention und Feed-Forward-Netzwerk bis hin zur Softmax-Ausgabeschicht. Darüber hinaus wurden die Konzepte des Pretrainings, Finetunings und der Quantisierung vorgestellt, die den lokalen Betrieb großer Sprachmodelle auf begrenzter Hardware ermöglichen.

Aufbauend auf diesen Grundlagen wird im nächsten Kapitel die Sprachverarbeitung (Speech-to-Text und Text-to-Speech) sowie das konkret eingesetzte System vorgestellt, das die beschriebene LLM-Technologie in eine vollständige Sprachpipeline einbettet.

# Kapitel 3

# Sprachverarbeitung: STT und TTS

Aufbauend auf den im vorherigen Kapitel beschriebenen Grundlagen der Transformer-Architektur wird nun die Sprachverarbeitung behandelt. Um die LLM-Technologie möglichst intuitiv in realen Anwendungen einsetzen zu können, war es in dieser Arbeit das Ziel, gesprochene Sprache in Text umzuwandeln und umgekehrt. Die Umwandlung von gesprochener Sprache in Text bezeichnet man als Speech-to-Text (STT) und die Umwandlung von Text in gesprochene Sprache als Text-to-Speech (TTS). Bevor diese Themen im Detail erläutert werden, wird zunächst eine physikalische Grundlage von Audiosignalen eingeführt, um die nachfolgenden Abschnitte besser einordnen zu können.

## 3.1 Physikalische Eigenschaften von Audiosignalen

Beim Sprechen versetzt ein Mensch die Luftmoleküle in Schwingung. Durch die Kompressibilität der Luft breiten sich diese als Schallwellen aus, welche physikalisch als orts- und zeitabhängige Schwankung des Luftdrucks $p$ um den atmosphärischen Ruhedruck ($1013,25\,\mathrm{hPa}$) beschrieben werden [11]. Die Einheit des Schalldrucks ist Pascal (Pa). Der Mensch nimmt Schallwellen primär durch die Klangfarbe (hoch/tief) und die Lautstärke wahr. Die physikalischen Äquivalente hierzu sind die Frequenz $f$ für die Klangfarbe und die Amplitude des Schalldrucks für die Lautstärke. Die Frequenz errechnet sich mit der Einheit Hertz (Hz) über die Kehrwert der Periodendauer $T$:

$$f = \frac{1}{T} \tag{3.1}$$

Ein Audiosignal, welches nur eine einzige Frequenzkomponente abdeckt (wie der musikalische Referenzton A4 mit 440 Hz), nennt man *Ton*. Eine Mischung aus Tönen mit ganzzahligen Frequenzverhältnissen (Grund- und Oberschwingungen) wird dagegen als *Klang* bezeichnet [12]. Die menschliche Sprache entsteht demnach aus einer Überlagerung

solcher harmonischer Schwingungen, wobei die relevanten Frequenzen typischerweise in einer Bandbreite zwischen 80 Hz und 12 kHz angesiedelt sind.

Da empirische Versuche gezeigt haben, dass das menschliche Lautstärkeempfinden logarithmisch funktioniert, ist die lineare Einheit Pascal zur Einordnung ungeeignet. Stattdessen zieht man den logarithmierten Schalldruckpegel $L$ (angegeben in Dezibel, dB) heran.

$$L = 20 \cdot \log_{10}\left(\frac{p}{p_0}\right) \text{ dB} \tag{3.2}$$

Der Faktor bringt den gemessenen Effektivwert des Schalldrucks $p$ ins Verhältnis zur menschlichen Hörschwelle $p_0$, welche bei $2 \cdot 10^{-5}$ Pa (entsprechend 0 dB bei 1 kHz) liegt. Der Vorfaktor von 20 wurde in der Definition bewusst so gewählt, dass eine Pegeldifferenz von 1 dB genau der relativen Unterschiedsschwelle des Gehörs entspricht, bei der zwei Schalldrücke gerade noch als unterschiedlich laut ermittelt werden können. Die Skala erstreckt sich dabei von der Hörschwelle bis hin zur Schmerzgrenze bei etwa 200 Pa, was einem extremen Lautstärkeeindruck von 140 dB entspricht.

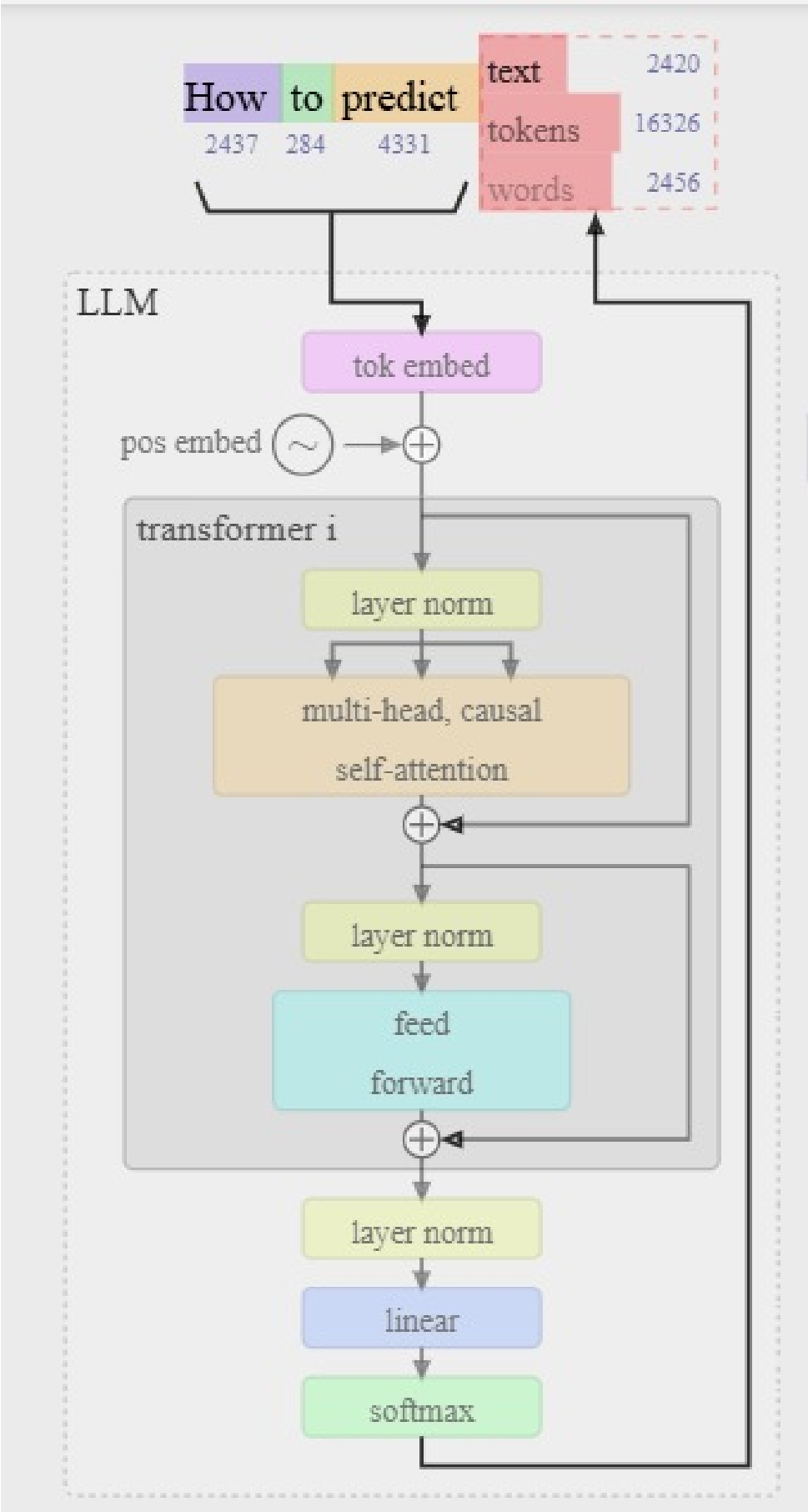


Abbildung 3.1: Datenfluss eines Large Language Models

Das Eingangssignal ist eine Audiodatei (typischerweise 16 kHz Mono). Um es für eine Tranformer-Architektur nutzbar zu machen, wird es in ein **Log-Mel-Spektrogramm** umgewandelt, eine Zeit-Frequenz-Darstellung.

### 3.1.1 Text-to-Speech (TTS)

Text-to-Speech bezeichnet die Erzeugung von Sprache aus Text. Moderne neuronale TTS-Systeme erzeugen natürlich klingende Sprache in einem mehrstufigen Prozess:

1. **Phonemisierung:** Der Eingabetext wird normalisiert und in eine phonetische Repräsentation (Phoneme) umgewandelt.
2. **Akustisches Modell:** Aus der Phonemsequenz wird ein Mel-Spektrogramm generiert eine Zeit-Frequenz-Darstellung, die Tonhöhe, Geschwindigkeit und Klangfarbe kodiert.
3. **Vocoder:** Das Mel-Spektrogramm wird in ein Audiosignal (Wellenform) umgewandelt.

End-to-End-Modelle wie **VITS** (Variational Inference with adversarial learning for end-to-end TTS) [13] kombinieren diese Stufen in einem einzigen Modell und werden direkt von Text zu Audio trainiert. Die Trainingsschleife minimiert eine Kombination aus Rekonstruktionsfehler und Adversarial Loss:

$$\mathcal{L}_{\text{VITS}} = \mathcal{L}_{\text{recon}} + \lambda \, \mathcal{L}_{\text{adv}} \tag{3.3}$$

## 3.2 Das eingesetzte System

Auf Basis der in den vorherigen Abschnitten beschriebenen Theorie wird in dieser Arbeit eine vollständige Sprachpipeline eingesetzt[14], die ausschließlich lokal ohne Cloud-Anbindung betrieben wird. Die drei Kernkomponenten sind:

**Spracherkennung: Whisper. Whisper** [15] ist ein von OpenAI veröffentlichtes Open-Source-Modell zur automatischen Spracherkennung. Es wurde auf 680.000 Stunden mehrsprachiger Audiodaten trainiert und basiert auf einer Transformer-Architektur. Whisper unterstützt mehrsprachige Erkennung. In dieser Arbeit wird Whisper lokal auf der IZPL-Hardware ausgeführt. Ein gesprochener Satz wie „Wie hoch ist die aktuelle Raumtemperatur?“ wird innerhalb weniger hundert Millisekunden transkribiert und als Zeichenkette an das Sprachmodell übergeben.

**Sprachmodell: LLaMA 7B. LLaMA 7B** [16] ist ein von Meta entwickeltes Open-Source-Sprachmodell mit 6,7 Milliarden Parametern. Es verwendet die in Abschnitt 2.2

beschriebene Transformer-Architektur mit folgenden Spezifikationen:

| Modell | $N$ | $d$ | $h$ | $d_{\text{ff}}$ | Parameter |
|---|---|---|---|---|---|
| GPT-2 (small) | 12 | 768 | 12 | 3.072 | 124 M |
| **LLaMA 7B** | **32** | **4.096** | **32** | **11.008** | **6,7 B** |
| LLaMA 13B | 40 | 5.120 | 40 | 13.824 | 13 B |
| GPT-3 | 96 | 12.288 | 96 | 49.152 | 175 B |

Tabelle 3.1: Architekturparameter – LLaMA 7B (fett) ist das in dieser Arbeit eingesetzte Modell.

Im Vergleich zum Standard-Transformer setzt LLaMA 7B folgende Anpassungen ein:

- **RMSNorm** (statt LayerNorm) vor jeder Operation (vgl. Gleichung 2.5)
- **SwiGLU**-Aktivierung im FFN (statt ReLU):

$$\text{FFN}_{\text{SwiGLU}}(x) = \big(\text{Swish}(xW_1) \odot (xW_3)\big)\, W_2, \quad \text{Swish}(z) = z \cdot \sigma(z) \tag{3.4}$$

- **RoPE** Positionskodierung (vgl. Gleichung 2.11)
- Maximale Kontextlänge: $T_{\max} = 4096$ Tokens

Das Modell wird im **Q4_K_M**-Format über `ollama` lokal betrieben, was einen VRAM-Bedarf von ca. 4 GB ergibt und den Betrieb auf der verfügbaren Hardware ohne relevante Qualitätseinbußen ermöglicht.

**Sprachsynthese: Piper TTS mit eigenem Stimmmodell. Piper TTS** [17] ist ein schnelles, Open-Source-TTS-System, das auf der VITS-Architektur (vgl. Abschnitt 3.1.1) basiert und für den lokalen, ressourcenschonenden Betrieb optimiert ist. In dieser Arbeit wurde ein eigenes Stimmmodell trainiert, um die Sprachausgabe natürlicher zu gestalten. Das Vorgehen folgt dem Verfahren von NetworkChuck [18]:

1. **Datenaufbereitung:** Aufnahme von Sprachsamples Die Samples werden rauschbereinigt, auf 22.050 Hz normalisiert und den zugehörigen Transkripten zugeordnet.
2. **Finetuning:** Training ausgehend von einem vortrainierten englischen Piper-Checkpoint. Ein Checkpoint repräsentiert die gelernten Parameter des neuronalen Netzes. Eine sogenannte Epoche beschreibt wie oft diese Parameter durch die Trainingsdaten weiter verfeinert wurden. Für das Training des eigenen Stimmmodells wurde ab der 6000 Epoche das Training bis zu 6250 fortgesetzt. Folglich wurden 250 mal die Sprachsamples benutzt um die Parameter des neuronalen Netzes zu verändern.

```
python3 -m piper_train \
```

```
  --dataset-dir ~/sprachdaten \
  --accelerator gpu --gpus 1 \
  --batch-size 32 --max_epochs 6000 \
  --resume_from_checkpoint base.ckpt \
  --precision 16 --quality medium
```

3. **ONNX-Export:** Das trainierte Modell wird in das ONNX-Format (Open Neural Network Exchange) exportiert, das eine hardwareunabhängige Inferenz ohne GPU ermöglicht:

```
python3 -m piper_train.export_onnx \
  checkpoint/epoch=5999.ckpt \
  output/stimme.onnx
```

Das resultierende Modell (`stimme.onnx`, ca. 60 MB) wird zusammen mit der Konfigurationsdatei `stimme.onnx.json` in Piper eingebunden und ermöglicht die Echtzeitsynthese auf der lokalen Hardware.

## 3.3 Zusammenfassung

Dieses Kapitel hat die Sprachverarbeitung im Kontext der eingesetzten Pipeline vorgestellt. Zunächst wurden die physikalischen Grundlagen von Audiosignalen erläutert, gefolgt von der Theorie hinter Text-to-Speech-Systemen. Anschließend wurde das konkret eingesetzte System beschrieben:

| Komponente | Eingesetztes Modell | Besonderheit |
|---|---|---|
| STT | Whisper | lokal, mehrsprachig, 80 Mel-Kanäle |
| LLM | LLaMA 7B (Q4_K_M, GGUF) | $N$=32, $d$=4096, ≈4 GB VRAM |
| TTS | Piper TTS + eigenes ONNX-Modell | VITS, $f_s$=22.050 Hz, kein Cloud |

Tabelle 3.2: Übersicht der eingesetzten Komponenten.

Mit dem Wissen über die eingesetzte Sprachpipeline werden im folgenden Kapitel die Hardwareanforderungen für den lokalen Betrieb eines LLMs und der konkrete Laboraufbau am IZPL beschrieben.

# Kapitel 4

# On-Premise-Infrastruktur und Hardware

Dieses Kapitel beschreibt die Hardwareanforderungen für den lokalen Betrieb eines Large Language Models und den konkreten Laboraufbau am IZPL. Um einordnenr zu können wie viele Rechenoperationen nötig sind für ein vorhergesagtes Token, wird zunächst anhand eines minimalen Beispielmodells (nano-GPT, 85.584 Parameter) gezeigt, um auf das in dieser Arbeit eingesetzte LLaMA 7B-Modell zu skalieren. Für den Vergleich der Rechenoperationen wird die Einheit FLOPs verwendet. Ein FLOP (Floating Point Operation) bezeichnet eine einzelne Gleitkomma-Rechenoperation, also eine Addition oder Multiplikation mit Dezimalzahlen. Zur besseren Übersicht werden Vielfache verwendet: 1 MFLOP = $10^6$ FLOPs (Mega), 1 GFLOP = $10^9$ FLOPs (Giga) und 1 TFLOP = $10^{12}$ FLOPs (Tera).

## 4.1 Rechenoperationen eines LLMs nano-GPT als Beispiel

Wie in Kapitel **??** bereits angeschnitten sind Large language models im ersten Schritt mit Embedding layers ausgestattet. Um die Skalierung allein dieses schrittes zu veranaschaulichen wie bereits genannt hat Gpt2 768 dimensionale Vektoren als Start vor jeder Matrixmultiplikation GPT3 jedoch hat für das 175 Milliarden Parameter modell 12288 dimensionen. **??** Um zu verstehen, warum LLMs hohe Hardware-Anforderungen s Brendan Bycroft stellt mit der interaktiven Visualisierung [19] ein nano-GPT-Modell mit **85.584 Parametern** zur Verfügung, das schrittweise von der Tokenisierung bis zum Softmax-Ausgan alle Operationen zeigt.

### 4.1.1 Modellkonfiguration

Das nano-GPT-Modell ist auf die simple AUfgbae ausgelegt, eine Buchstabensequenz zu sortieren (z. B. `C B A B B C` → `A B B B C C`). Die Modellparameter sind:

| Parameter | Bezeichnung | Wert |
|---|---|---|
| $\lvert\mathcal{V}\rvert$ | Vokabulargröße (Zeichen) | 65 |
| $T$ | Kontextlänge (Tokens) | 64 |
| $C$ | Embedding-Dimension | 48 |
| $L$ | Anzahl Transformer-Blöcke | 3 |
| $H$ | Anzahl Attention-Heads | 3 |
| $C/H$ | Head-Dimension | 16 |
| $4C$ | MLP innere Dimension | 192 |

Tabelle 4.1: Konfiguration des nano-GPT-Modells [19].

### 4.1.2 Rechenoperationen und Skalierung

Die interaktive Visualisierung von Brendan Bycroft [19] zeigt jede einzelne Addition und Multiplikation im Forward Pass des nano-GPT-Modells. Die 85.584 Parameter verteilen sich auf Token- und Positions-Embedding, drei Transformer-Blöcke (jeweils mit Attention und MLP) sowie eine finale Normierungsschicht.

Für eine Eingabesequenz der Länge $T = 6$ dominieren die MLP-Schichten den Rechenaufwand mit $\approx 76\,\%$ der FLOPs, während die Attention-Berechnungen bei kurzen Sequenzen vergleichsweise gering ausfallen. Insgesamt ergibt sich ein Aufwand von $\approx 1{,}75$ MFLOPs pro Forward Pass.

**Skalierung auf LLaMA 7B.** Dieselbe Struktur gilt für LLaMA 7B, jedoch mit deutlich größeren Dimensionen ($C = 4096$, $L = 32$, $H = 32$, $T_{\max} = 4096$). Die Anzahl der Floating-Point-Operationen pro Token lässt sich grob abschätzen als:

$$\text{FLOPs/Token} \approx 2 \times N_{\text{Parameter}} = 2 \times 6{,}7 \times 10^9 \approx 13{,}4 \text{ GFLOPs} \tag{4.1}$$

Diese Faustformel [20] gilt für Autoregressive Inferenz, bei der $T \ll C$ gilt. Bei einer Antwort mit 200 Token ergibt das:

$$200 \times 13{,}4 \text{ GFLOPs} = 2{,}68 \text{ TFLOPs} \tag{4.2}$$

## 4.2 Motivation für On-Premise-Betrieb

Wie in Kapitel 1 dargelegt, sprechen insbesondere Datenschutz, Compliance und die Abhängigkeit von US-Cloud-Anbietern für einen lokalen Betrieb von Sprachmodellen. Über diese bereits diskutierten Aspekte hinaus sind für den Einsatz auf Edge-Geräten und in der Robotik zwei weitere Faktoren entscheidend:

- **Verfügbarkeit und Latenz:** Cloud-Dienste sind von Internetzugang und der Verfügbarkeit des Anbieters abhängig. Lokale Modelle können in Echtzeit und ohne Netzwerkverzögerung reagieren – relevant für robotische Anwendungen.
- **Anpassbarkeit:** Lokale Modelle können feinabgestimmt, quantisiert und mit systemspezifischem Kontext (z. B. RAG) versehen werden, ohne von Anbieterbeschränkungen abhängig zu sein.

Tabelle 4.2 fasst die wesentlichen Unterschiede zwischen Cloud- und On-Premise-Betrieb zusammen.

| Kriterium | Cloud (API) | On-Premise |
|---|---|---|
| Datenschutz | Daten verlassen das Netz | vollständig lokal |
| Latenz | 100–2000 ms (Netz + Server) | 50–500 ms (lokal) |
| Verfügbarkeit | abhängig von Anbieter | lokal kontrollierbar |
| Kosten | pro Token (variabel) | Einmalinvestition |
| Wartung | keine | selbstverantwortlich |
| Modellwahl | begrenzt (Anbieter-API) | frei wählbar |

Tabelle 4.2: Vergleich Cloud-API vs. On-Premise-Betrieb.

## 4.3 Hardwareanforderungen

Aus den in Abschnitt 4.1 beschriebenen Rechenoperationen und der Skalierung auf LLaMA 7B ergeben sich konkrete Hardware-Anforderungen.

### 4.3.1 GPU und VRAM

Die GPU ist die kritische Ressource für LLM-Inferenz. Zwei Faktoren sind maßgebend:

1. **VRAM:** Die gesamten Modellgewichte müssen im GPU-Speicher liegen. Für LLaMA 7B im Format Q4_K_M sind das ca. 4–5 GB (vgl. Tabelle 2.1). Zusätzlich werden die KV-Cache-Aktivierungen für den Kontext benötigt:

$$\text{KV-Cache} \approx 2 \times L \times T \times C \times \text{Bytes/Wert} \tag{4.3}$$

Für LLaMA 7B bei $T = 2048$, $L = 32$, $C = 4096$ und FP16: $2 \times 32 \times 2048 \times 4096 \times 2 \approx$ 1,07 GB.

2. **Speicherbandbreite:** Bei der Autoregressive Inferenz wird pro Token die gesamte Gewichtsmatrix einmal gelesen – Memory-Bound-Betrieb. Eine hohe Speicherbandbreite (GB/s) ist entscheidender als rohe FLOP-Leistung.

| GPU | VRAM | Bandbreite | LLaMA 7B Q4 |
|---|---|---|---|
| NVIDIA RTX 3060 | 12 GB | 360 GB/s | ✓ |
| NVIDIA RTX 3090 | 24 GB | 936 GB/s | ✓✓ |
| NVIDIA RTX 4090 | 24 GB | 1.008 GB/s | ✓✓ |
| AMD RX 7900 XTX | 24 GB | 960 GB/s | ✓✓ |
| NVIDIA A100 (80 GB) | 80 GB | 2.000 GB/s | ✓✓✓ |

Tabelle 4.3: Gängige GPUs für lokale LLM-Inferenz. ✓ : ausreichend, ✓✓ : gut geeignet, ✓✓✓ : optimal.

### 4.3.2 CPU und Arbeitsspeicher

Falls kein ausreichender VRAM vorhanden ist, können Modellschichten auf den CPU-RAM ausgelagert werden (**CPU Offloading**). Dies senkt die VRAM-Anforderungen drastisch, reduziert aber den Durchsatz deutlich (typisch 1–3 Token/s statt 10–30 Token/s).

Mindestanforderungen für LLaMA 7B Q4 mit CPU-Offloading:

- **RAM:** $\geq$ 16 GB (Modell: $\approx$ 4 GB + Betriebssystem + KV-Cache)
- **CPU:** Multi-Core mit hoher Speicherbandbreite (z. B. AMD Ryzen 9 oder Intel Core i9)

### 4.3.3 Speichersystem

Das Laden eines GGUF-Modells (ca. 4 GB) vom Speichermedium beeinflusst die Startlatenz. Mit einer NVMe-SSD (3.000–7.000 MB/s) beträgt die Ladezeit $\approx$ 1–2 Sekunden, mit einer HDDs (100–200 MB/s) $\approx$ 20–40 Sekunden.

## 4.4 Laboraufbau am IZPL

Der konkrete Hardware-Aufbau für diese Arbeit befindet sich im Labor des **IZPL** (Innovationszentrum für Produktionsund Logistiklösungen) der OTH Regensburg. Das System umfasst:

| Komponente | Spezifikation |
|---|---|
| Prozessor (CPU) | |
| Arbeitsspeicher | |
| Grafikkarte (GPU) | |
| Speicher | |
| Betriebssystem | |
| LLM-Laufzeitumgebung | `ollama` |
| Modell | LLaMA 7B (Q4_K_M, GGUF) |
| STT | Whisper (lokal) |
| TTS | Piper TTS (ONNX, lokal) |

Tabelle 4.4: Hardware- und Software-Konfiguration am IZPL.

Mit dieser Konfiguration werden folgende Betriebswerte erreicht:

- **Inferenzgeschwindigkeit:**
- **Latenz (Erstantwort):**
- **VRAM-Auslastung:**

## 4.5 Zusammenfassung

Dieses Kapitel hat den Rechenbedarf von LLMs anhand des nano-GPT-Modells (85.584 Parameter, [19]) veranschaulicht und auf LLaMA 7B skaliert. Die kritischen Hardware-Ressourcen für lokale LLM-Inferenz sind **VRAM** (Modellgewichte + KV-Cache, vgl. Tabelle 2.1) und **Speicherbandbreite** (Memory-Bound-Betrieb). LLaMA 7B im Q4_K_M-Format kann mit einer modernen Consumer-GPU ($\geq$ 6 GB VRAM) lokal betrieben werden. Die IZPL-Hardware erfüllt diese Anforderungen und ermöglicht den On-Premise-Betrieb der vollständigen Sprachpipeline ohne Cloud-Anbindung.

Nachdem die theoretischen und infrastrukturellen Grundlagen gelegt sind, folgt im nächsten Kapitel die Evaluation des Systems im praktischen Betrieb.

# Kapitel 5

# Evaluation

Diese Kapitel führt eine Versuchsreihe durch welche mit dem im IZPL vorhanden Hardwaresetup durhc. Als LLM wurde 7Milliarden Paramter mit Piper als Text to Speach Modell. Das Githun Repository **??** dient hierbei als Startgerüst und wurde nur bzgl. geringen Änderungen verändert.

## 5.1 Methodik der Evaluation

Es werden die Durchschnittliche Antwortzeit die WER und die Qualität der Antwort verglichen. Zur Beurteilung der Hardwareanforderungen wird zudem der VRAM-Bedarf betrachtet.

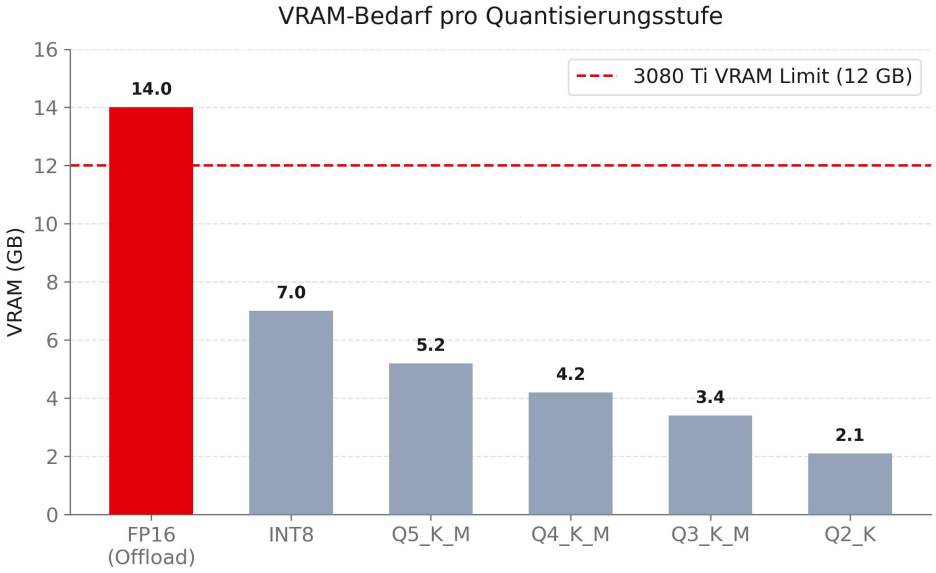


Abbildung 5.1: Simulierter VRAM-Bedarf je Quantisierungsstufe im Vergleich zum 12-GB-Limit der RTX 3080 Ti

## 5.2 Antwortzeiten und Performanz

Die Performanz des Modells wird primär über die Token-Generierungsgeschwindigkeit sowie die initiale Latenz (Time-to-First-Token) gemessen.

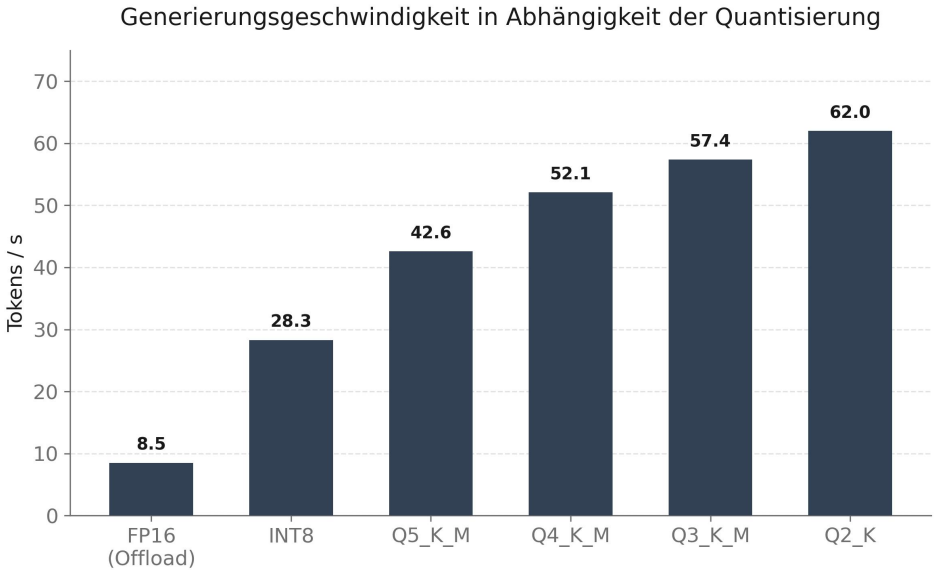


Abbildung 5.2: Generierungsgeschwindigkeit (Tokens/s) in Abhängigkeit der Quantisierung

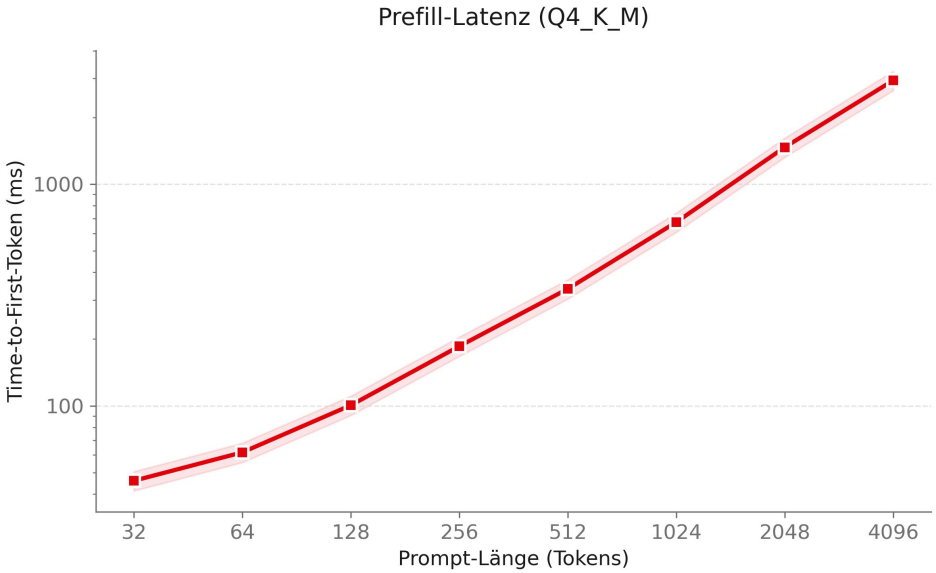


Abbildung 5.3: Prefill-Latenz (Time-to-First-Token) über verschiedene Prompt-Längen (Q4_K_M)

Ein detaillierter Blick auf den Generierungsprozess im zeitlichen Verlauf zeigt die kontinuierliche Token-Ausgabe sowie die dynamische VRAM-Auslastung während der Inferenz.

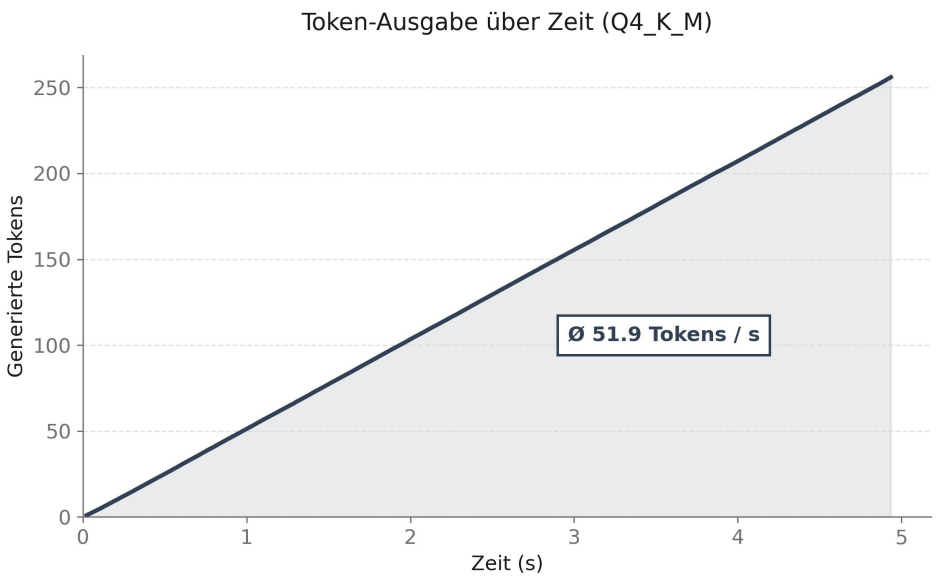


Abbildung 5.4: Kumulative Token-Ausgabe über die Zeit bei der 4-Bit-Quantisierung

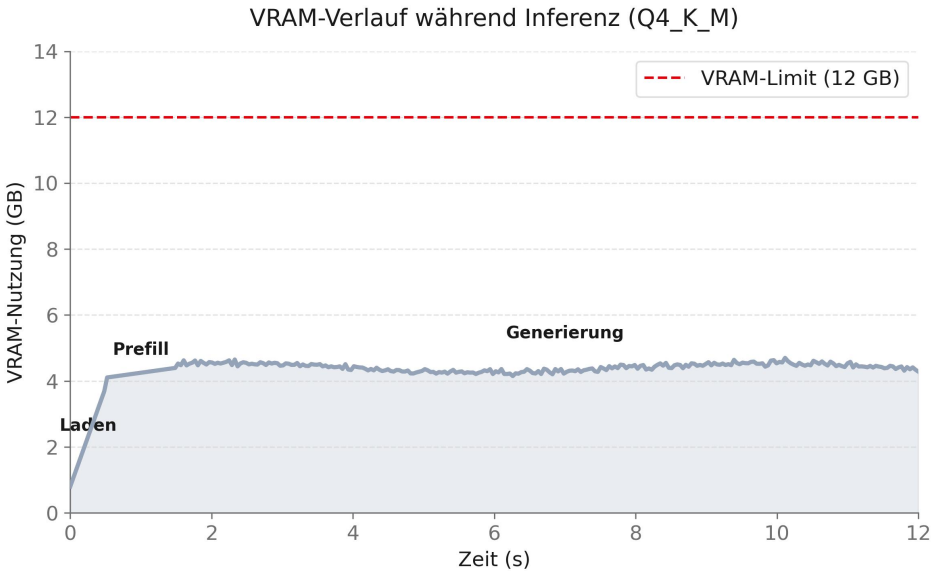


Abbildung 5.5: Zeitlicher VRAM-Verlauf während der Modell-Ausführung gegliedert in Laden, Prefill und Generierung

## 5.3 Qualität der Konversation

Für LLMs gibt es viele Benchmarks zu den aus objektiver sicht zählenden gehört die sogenannte Perplexity. Durch die Quantisierung des Modells lässt sich eine minimale Einbuße bei der Sprachqualität beobachten, welche jedoch bei 4-Bit-Gewichten noch als praxistauglich gilt.

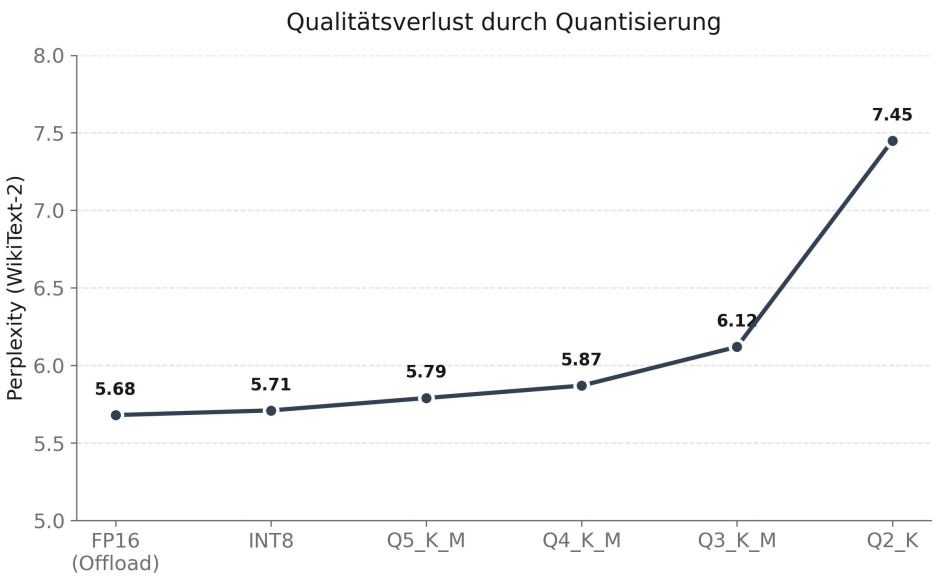


Abbildung 5.6: Qualitätsverlust (Perplexity auf WikiText-2) durch Quantisierungsverfahren

Die hier gewonnenen Evaluationsergebnisse werden im folgenden Kapitel um Sicherheits- und Fehlerbetrachtungen ergänzt, die für den produktiven Einsatz in der Robotik entscheidend sind.

# Kapitel 6

# Fazit und Ausblick

Zusammenfassend ist zu sagen dass On premise lösungen welche lokal auf server betrieben werden in bestimmmten anwendungsfällen eine sehr gut alternative zu cloud basierten lösungen bieten. Robotikanwendungen in innerbetrieblichen Bereich sind oftmals überschaubar genung bezogen auf veränderlichen variabeln, dass die anpassung der hyperparamter eines klein bis mittlgroßen modells überschaubar sind und demnach gut lokal betrieben werden können. Für große Modelle wie die meißsten sie heute kennen sind große datenzentren nötig mit hohen energiekosten welche für einzelen anwendungen nur wenig sinn ergeben. Dennoch ist immer abzugwwägen ob es in Zukunft für ein wachsendes Unternehmen nicht von Vorteil wäre geld in die hand zu nehmen um eine Serverstruktur zu schaffen welche sich vom Cloud geschäft ein stück weit loslöst und eigen LLM auf eigenen Servern betreibt. Zudem wird trotz wachsender Compute möglichkeiten eine linie nicht überschritten werden können laut **??**

## 6.1 Zusammenfassung der Ergebnisse

## 6.2 Ausblick auf die Masterarbeit

Die Masterarbeit knüpft an vielen der hier erarbeiteten Grundlagen direkt an und umfasst weiterfürhende Themen wie vision language action models in der Robotik. Zusammene mit einer weiteren Forschungs und Entwicklungsarbeit zum Thema ROS2 auf einem IGus Rebel 6 DOf wird ein Testaufbau entwickelt welcher konkret Robotik und Large Languange models im Kontext von Open source und on premise evaluiert. Generell bleit abzuwarten ob der hype um diese neue Technologie den erhofften wandel der gesellschaft standhalten kann oder doch kompromisse eingegangen werden müssen. auf jden fall wurde durch diese Forschungsarbeit ein Grundstein zur Erklärung der Modelle gelegt.

# Anhang A

# Anhang

## A.1 Konfigurationsdateien

## A.2 Testprotokolle

## A.3 Zusätzliche Abbildungen

# Literaturverzeichnis


[1] Jens-Christoph Brendel. *Musks xAI hat bislang größten KI-Supercomputer aller Zeiten gebaut.* 2025. URL: https://www.linux-magazin.de/news/musks-xai-hat-bislang-groessten-ki-supercomputer-aller-zeiten-gebaut/ (besucht am 16. 03. 2026).

[2] Kif Leswing. *Huang's 'half a trillion' Nvidia forecast will come up at Q3 earnings.* 2025. URL: https://www.cnbc.com/2025/11/17/huangs-half-a-trillion-nvidia-forecast-will-come-up-at-q3-earnings.html (besucht am 16. 03. 2026).

[3] Matthias Janson. *Wie viel Strom brauchen Rechenzentren in Zukunft?* 2025. URL: https://de.statista.com/infografik/34392/weltweiter-stromverbrauch-von-datenzentren-nach-ausstattungsmerkmalen/ (besucht am 16. 03. 2026).

[4] Bitkom e.V. *Cloud Report 2025.* 2025. URL: https://www.bitkom.org/Presse/Presseinformation/Cloud-Report-2025 (besucht am 16. 03. 2026).

[5] Daniel Jurafsky und James H. Martin. *Speech and Language Processing.* 3rd. 2026. URL: https://web.stanford.edu/~jurafsky/slp3/.

[6] Tom B. Brown u. a. *Language Models are Few-Shot Learners.* 2020. arXiv: 2005.14165 [cs.CL]. URL: https://arxiv.org/abs/2005.14165.

[7] Rico Sennrich, Barry Haddow und Alexandra Birch. *Neural Machine Translation of Rare Words with Subword Units.* 2016. arXiv: 1508.07909. URL: https://arxiv.org/abs/1508.07909.

[8] Ashish Vaswani u. a. *Attention Is All You Need.* 2017. arXiv: 1706.03762. URL: https://arxiv.org/abs/1706.03762.

[9] Jianlin Su u. a. *RoFormer: Enhanced Transformer with Rotary Position Embedding.* 2021. arXiv: 2104.09864. URL: https://arxiv.org/abs/2104.09864.

[10] Hao Chen u. a. *Inference performance evaluation for LLMs on edge devices with a novel benchmarking framework and metric.* 2025. arXiv: 2508.11269 [cs.PF]. URL: https://arxiv.org/abs/2508.11269.

[11] Michael Möser. *Technische Akustik.* Berlin, Heidelberg: Springer Vieweg, 2015.

[12] Gerhard Müller und Michael Möser. *Handbuch der Technischen Akustik.* Berlin, Heidelberg: Springer Vieweg, 2015.

[13] Jaehyeon Kim, Jungil Kong und Juhee Son. *Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech*. 2021. arXiv: 2106.06103. URL: https://arxiv.org/abs/2106.06103.

[14] David Ng. *GLaDOS: The Personality Core*. GitHub repository. 2024. URL: https://github.com/dnhkng/GLaDOS.

[15] Alec Radford u. a. *Robust Speech Recognition via Large-Scale Weak Supervision*. 2022. arXiv: 2212.04356. URL: https://arxiv.org/abs/2212.04356.

[16] Hugo Touvron u. a. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971. URL: https://arxiv.org/abs/2302.13971.

[17] Michael Hanson. *Piper: A fast, local neural text to speech system*. 2023. URL: https://github.com/rhasspy/piper.

[18] NetworkChuck. *Clone Your Own Voice with AI*. 2024. URL: https://www.youtube.com/watch?v=placeholder.

[19] Brendan Bycroft. *LLM Visualization*. 2023. URL: https://bbycroft.net/llm.

[20] Jared Kaplan u. a. *Scaling Laws for Neural Language Models*. 2020. arXiv: 2001.08361. URL: https://arxiv.org/abs/2001.08361.